Solver 거래 처리 및 수수료 시스템 분석

개요

Solver 서비스의 거래 처리 및 수수료 시스템은 크로스체인 주문을 효율적으로 처리하고, 적절한 수수료를 부과하여 수익성을 보장하는 핵심 메커니즘입니다. 이 시스템은 가격 오라클, 유동성 관리, 리스크 평가를 통합하여 최적화된 거래 실행을 제공합니다.

거래 처리 플로우

1. 주문 검증 시스템 (`solver/app/check.go`)

```
func newChecker(backends unibackend.Backends, isAllowedCall callAllowFunc,
priceFunc priceFunc, solverAddr, outboxAddr common.Address) checkFunc {
return func(ctx context.Context, req types.CheckRequest) error {
// 1. 체인 지원 여부 확인
if _, err := backends.Backend(req.SourceChainID); err != nil {
return newRejection(types.RejectUnsupportedSrcChain, ...)
}
// 2. 토큰 파싱 및 검증
deposit, err := parseTokenAmt(req.SourceChainID, req.Deposit)
expenses, err := parseExpenses(req.DestinationChainID, req.Expenses, req.Calls)
// 3. 가격 및 견적 확인
if err := checkQuote(ctx, priceFunc, []TokenAmt{deposit}, expenses); err != ni
```

```
return err
}
// 4. 유동성 확인
if err := checkLiquidity(ctx, expenses, dstBackend, solverAddr); err != nil {
return err
}
// 5. 호출 허용 여부 확인
if err := checkCalls(req.DestinationChainID, req.Calls, isAllowedCall); err !=
return err
}
// 6. 승인 상태 확인
if err := checkApprovals(ctx, expenses, dstBackend, solverAddr, outboxAddr); e
return err
}
// 7. 실제 실행 시뮬레이션
return checkFill(ctx, dstBackend, orderID, fillOriginData, ...)
}
}
```

2. 주문 거부 시스템 (`solver/app/reject.go`)

거부 사유 분류

```go
type RejectReason uint8


const (
RejectNone RejectReason = 0  // 거부 없음
RejectDestCallReverts RejectReason = 1  // 목적지 호출 실패
RejectInvalidDeposit RejectReason = 2  // 잘못된 예치
RejectInvalidExpense RejectReason = 3  // 잘못된 비용
RejectInsufficientDeposit RejectReason = 4  // 예치 부족
RejectInsufficientInventory RejectReason = 5  // 재고 부족
RejectUnsupportedDeposit RejectReason = 6  // 지원하지 않는 예치 토큰
RejectUnsupportedExpense RejectReason = 7  // 지원하지 않는 비용 토큰
RejectUnsupportedDestChain RejectReason = 8  // 지원하지 않는 목적지 체인
RejectUnsupportedSrcChain RejectReason = 9  // 지원하지 않는 소스 체인
RejectExpenseOverMax RejectReason = 11  // 비용 한도 초과
RejectExpenseUnderMin RejectReason = 12  // 비용 최소값 미달
RejectCallNotAllowed RejectReason = 13  // 허용되지 않는 호출
)
```

유동성 검증

```go
func  checkLiquidity(ctx context.Context, expenses []TokenAmt, backend unibacken
for  _, expense := range  expenses {
// 현재 잔액 조회
```

```
bal, err := tokenutil.UniBalanceOf(ctx, backend, expense.Token, uni.EVMAddress(s

// 최소 안전 잔액 설정 (ETH의 경우 가스비 고려)

minSafe := bi.Zero()

if  expense.Token.Is(tokens.ETH) {

minSafe = minSafeETH  // 0.05 ETH

}

// 잔액 부족 시 거부

if  bi.LT(bal, bi.Add(expense.Amount, minSafe)) {

return  newRejection(types.RejectInsufficientInventory, ...)

}

}

return  nil

}
```

가격 시스템

1. 가격 오라클 (`solver/app/pricer.go`)

```
type priceFunc func(ctx context.Context, deposit, expense tokens.Token) (types.P



func  newPriceFunc(pricer tokenpricer.Pricer) priceFunc {

return  func(ctx context.Context, deposit, expense tokens.Token) (types.Price, e

// 동일 토큰인 경우 1:1 비율
```

```go
price := big.NewRat(1, 1)
if !areEqualBySymbol(deposit, expense) {
var  err  error
price, err = pricer.Price(ctx, deposit.Asset, expense.Asset)
if  err != nil {
return types.Price{}, err
}
}
return types.Price{
Price: price,
Deposit: deposit.Asset,
Expense: expense.Asset,
}, nil
}
}
```

2. 가격 구조체 (solver/types/price.go)

```go
type Price struct {
Price *big.Rat // 가격 비율 (expense/deposit)
Deposit tokens.Asset // 예치 토큰
Expense tokens.Asset // 비용 토큰
```

```go
}


// 수수료 적용
func (p Price) WithFeeBips(fee  int64) Price {
clone := p
if  fee > 0 {
// 수수료는 받는 토큰(expense)을 줄이는 방식으로 적용
feeRat := big.NewRat(10_000, 10_000+fee)
clone.Price = new(big.Rat).Mul(clone.Price, feeRat)
}
return  clone
}


// 예치 금액을 비용 금액으로 변환
func (p Price) ToExpense(depositAmount *big.Int) *big.Int {
rebasedDeposit := bi.Rebase(depositAmount, p.Deposit.Decimals, p.Expense.Decimal
return  bi.MulRatFloor(rebasedDeposit, p.Price)
}


// 비용 금액을 예치 금액으로 변환
func (p Price) ToDeposit(expenseAmount *big.Int) *big.Int {
rebasedExpense := bi.Rebase(expenseAmount, p.Expense.Decimals, p.Deposit.Decimal
return  bi.MulRatCeil(rebasedExpense, p.Inverse().Price)
```

```
}
```

수수료 시스템

1. 수수료 구조 (solver/app/quote.go)

```
// 표준 수수료: 0.3% (30 basis points)
const standardFeeBips = 30

// 토큰 쌍별 수수료 계산
func feeBips(a, b tokens.Asset) int64 {
// OMNI<>OMNI: 수수료 없음
if a == tokens.OMNI && b == tokens.OMNI {
return 0
}
// NOM<>NOM: 수수료 없음
if a == tokens.NOM && b == tokens.NOM {
return 0
}
// NOM<>OMNI: 수수료 없음 (생태계 토큰 간 거래 장려)
if (a == tokens.NOM && b == tokens.OMNI) || (a == tokens.OMNI && b == tokens.NOM
return 0
}
```

```
// 기타 모든 경우: 표준 수수료 적용
return standardFeeBips
}
```

2. 견적 시스템

```
func newQuoter(priceFunc priceFunc) quoteFunc {
return func(ctx context.Context, req types.QuoteRequest) (types.QuoteResponse,
// 예치 또는 비용 중 하나만 0이어야 함
isDepositQuote := bi.IsZero(req.Deposit.Amount)
isExpenseQuote := bi.IsZero(req.Expense.Amount)
if isDepositQuote == isExpenseQuote {
return returnErr(http.StatusBadRequest, "deposit and expense amount cannot be b
}
// 가격 조회
price, err := priceFunc(ctx, depositTkn, expenseTkn)
if err != nil {
return types.QuoteResponse{}, newAPIError(err, http.StatusBadRequest)
}
// 수수료 적용
price = price.WithFeeBips(feeBips(depositTkn.Asset, expenseTkn.Asset))
// 견적 계산
if isDepositQuote {
```

```
req.Deposit.Amount = price.ToDeposit(req.Expense.Amount)
} else {
req.Expense.Amount = price.ToExpense(req.Deposit.Amount)
}
return types.QuoteResponse{
Deposit: req.Deposit,
Expense: req.Expense,
}, maybeMinMaxReject(req.Expense.Amount, expenseTkn)
}
}
```

토큰 관리 시스템

1. 지원 토큰 (`solver/app/tokens.go`)

```
var  supportedAssets = map[tokens.Asset]bool{
tokens.ETH: true, // Ethereum
tokens.OMNI: true, // Omni 네이티브 토큰
tokens.NOM: true, // Nomad 토큰
tokens.WSTETH: true, // Wrapped Staked ETH
tokens.USDC: true, // USD Coin
tokens.USDT: true, // Tether USD
tokens.WETH: true, // Wrapped ETH
```

```
tokens.METH: true, // Mantle ETH

tokens.MNT: true, // Mantle 토큰

tokens.HYPE: true, // Hyperliquid 토큰

tokens.USDT0: true, // USDT on Hyperliquid

}
```

2. 거래 한도 설정

```
type SpendBounds struct {

MinSpend *big.Int // 최소 거래 금액

MaxSpend *big.Int // 최대 거래 금액

}


var  tokenSpendBounds = map[tokens.Asset]map[tokens.ChainClass]SpendBounds{

tokens.ETH: {

tokens.ClassMainnet: {

MinSpend: bi.Ether(0.001), // 0.001 ETH

MaxSpend: bi.Ether(6), // 6 ETH

},

tokens.ClassTestnet: {

MinSpend: bi.Ether(0.001), // 0.001 ETH

MaxSpend: bi.Ether(3), // 3 ETH

},
```

```
},
tokens.USDC: {
tokens.ClassMainnet: {
MinSpend: bi.Dec6(0.1), // 0.1 USDC
MaxSpend: bi.Dec6(10_000), // 10,000 USDC
},
},
}
```

3. 체인별 특별 한도

```
var  chainOverrides = map[tokens.Asset]map[uint64]SpendBounds{
tokens.ETH: {
evmchain.IDEthereum: {
MinSpend: bi.Ether(0.001), // 0.001 ETH
MaxSpend: bi.Ether(65), // 65 ETH (메인넷에서 더 높은 한도)
},
},
tokens.USDC: {
evmchain.IDEthereum: {
MinSpend: bi.Dec6(0.1), // 0.1 USDC
MaxSpend: bi.Dec6(100_000), // 100,000 USDC (메인넷에서 더 높은 한도)
},
```

```
},

}
```

유동성 임계값 관리

1. 임계값 구조 (`solver/fundthresh/thresholds.go`)

```
type FundThreshold struct {

token tokens.Token

min  float64  // 경고 임계값 (이하 시 알림)

target  float64  // 목표 잔액 (이하 시 부족으로 간주)

surplus  float64  // 잉여 임계값 (이상 시 잉여로 간주)

minSwap  float64  // 최소 스왑 금액

maxSwap  float64  // 최대 스왑 금액

}
```

2. 체인별 임계값 설정

```
var  thresholds = map[tokens.Token]FundThreshold{

// Ethereum 메인넷 ETH

mustToken(evmchain.IDEthereum, tokens.ETH): {

min: 20, // 20 ETH 이하 시 경고

target: 50, // 50 ETH 목표 잔액

surplus: 60, // 60 ETH 이상 시 잉여
```

```
minSwap: 1, // 최소 1 ETH 스왑
maxSwap: 5, // 최대 5 ETH 스왑
},
// Ethereum 메인넷 USDC
mustToken(evmchain.IDEthereum, tokens.USDC): {
min: 50000, // 50,000 USDC 이하 시 경고
target: 100000, // 100,000 USDC 목표 잔액
surplus: 120000, // 120,000 USDC 이상 시 잉여
minSwap: 1000, // 최소 1,000 USDC 스왑
maxSwap: 10000, // 최대 10,000 USDC 스왑
},
}
```

손익 관리 시스템

1. 손익 추적 (`solver/app/pnl.go`)

```
func  newFilledPnlFunc(
pricer tokenpricer.Pricer,
targetName targetFunc,
namer  func(uint64) string,
destFilledAge destFilledAge,
) filledPnLFunc {
```

```go
return func(ctx context.Context, order Order, rec *ethclient.Receipt) error {
// 1. 비용 로깅 (지출)
for _, tknAmt := range maxSpent {
p := pnl.LogP{
Type: pnl.Expense,
AmountGwei: toGweiF64(tknAmt.Amount, tknAmt.Token.Decimals),
Currency: pnl.Currency(tknAmt.Token.Symbol),
Category: "solver_expense",
Subcategory: target,
Chain: dstChainName,
ID: order.ID.String(),
}
pnl.Log(ctx, p)
usdPnL(ctx, pricer, tknAmt.Token.Asset, p)
}
// 2. 수익 로깅 (수입)
for _, tknAmt := range minReceived {
p := pnl.LogP{
Type: pnl.Income,
AmountGwei: toGweiF64(tknAmt.Amount, tknAmt.Token.Decimals),
Currency: pnl.Currency(tknAmt.Token.Symbol),
Category: "solver_deposit",
Subcategory: target,
```

```
Chain: srcChainName,
ID: order.ID.String(),
}
pnl.Log(ctx, p)
usdPnL(ctx, pricer, tknAmt.Token.Asset, p)
}
return nil
}
}
```

2. 가스비 추적

```
func gasPnL(
ctx context.Context,
pricer tokenpricer.Pricer,
chainID uint64,
chainName string,
rec *ethclient.Receipt,
subCat string,
id string,
) error {
// 기본 가스비 계산
amount := bi.MulRaw(rec.EffectiveGasPrice, rec.GasUsed)
```

```
// Optimism L1 수수료 추가
if  rec.OPL1Fee != nil {
amount = bi.Add(amount, rec.OPL1Fee)
}
// 크로스체인 호출 수수료 추가
if  fee, ok := maybeParseXCallFee(rec);  ok {
amount = bi.Add(amount, fee)
}
// 가스비를 비용으로 로깅
p := pnl.LogP{
Type: pnl.Expense,
AmountGwei: bi.ToGweiF64(amount),
Currency: pnl.Currency(nativeToken.Symbol),
Category: "gas",
Subcategory: subCat,
Chain: chainName,
ID: id,
}
pnl.Log(ctx, p)
usdPnL(ctx, pricer, nativeToken.Asset, p)
return  nil
}
```

거래 최적화 알고리즘

1. 가격 슬리피지 모니터링

```
func debugOrderPrice(ctx context.Context, priceFunc priceFunc, order Order) {
// 주문 가격 계산
orderPrice := types.Price{
Price: new(big.Rat).SetFrac(expenseAmt, depositAmt),
Deposit: depositTkn.Asset,
Expense: expenseTkn.Asset,
}
// 현재 시장 가격 조회
currentPrice, err := priceFunc(ctx, depositTkn, expenseTkn)
if err != nil {
return
}
// 수익률 계산 (basis points)
currentF64, _ := currentPrice.Price.Float64()
orderF64, _ := orderPrice.Price.Float64()
profitBips := math.Round((currentF64 - orderF64) / orderF64 * 10_000)
slippageBips := int64(profitBips) - feeBips(depositTkn.Asset, expenseTkn.Asset)
log.Debug(ctx, "Pending order amounts",
"deposit", depositTkn.FormatAmt(depositAmt),
```

```
"expense", expenseTkn.FormatAmt(expenseAmt),
"slippage_bips", slippageBips,
"price_current", currentPrice.FormatF64(),
"price_pair", fmt.Sprintf("%s/%s", orderPrice.Expense, orderPrice.Deposit),
)
}
```

2. 동적 가격 모니터링

```
func monitorPricesForever(ctx context.Context, priceFunc priceFunc) {
timer := time.NewTimer(0)
defer timer.Stop()
for {
select {
case <-ctx.Done():
return
case <-timer.C:
timer.Reset(time.Minute) // 1분마다 가격 업데이트
for asset := range supportedAssets {
err := monitorPriceOnce(ctx, asset, tokens.USDC, priceFunc)
if err != nil {
log.Warn(ctx, "Failed monitoring price (will retry)", err, "asset", asset)
break
```

```
}
}
}
}
}
```

리스크 관리

1. 호출 제한 시스템

```
func checkCalls(destChainID uint64, calls []types.Call, isAllowed callAllowFun
for _, call := range calls {
if !isAllowed(destChainID, call.Target, call.Data) {
return newRejection(types.RejectCallNotAllowed,
errors.New("call not allowed",
"target", call.Target.Hex(),
"data", hexutil.Encode(call.Data)))
}
}
return nil
}
```

2. 실행 시뮬레이션

```go
func  checkFill(
ctx context.Context,
backend unibackend.Backend,
orderID OrderID,
fillOriginData []byte,
nativeValue *big.Int,
solverAddr, outboxAddr common.Address,
sameChain  bool,
) error {
// 실제 실행 시뮬레이션
msg, err := fillCallMsg(ctx, client, orderID, fillOriginData, nativeValue, solve
if  err != nil {
return  errors.Wrap(err, "fill call msg")
}
returnData, err := client.CallContract(ctx, msg, nil)
if  err == nil {
return  nil  // 실행 성공
}
// 실행 실패 시 상세 분석
if !sameChain {
return &RejectionError{
Reason: types.RejectDestCallReverts,
```

```
Err: errors.Wrap(err, "call contract",
"return_data", hexutil.Encode(returnData),
"solidity_err", solvernet.DetectCustomError(err)),
}
}
return  nil
}
```

성능 최적화

1. 병렬 처리

- **체인별 독립 처리**: 각 체인의 이벤트를 독립적으로 처리
- **비동기 작업 큐**: 주문 처리를 비동기로 실행
- **배치 승인**: 여러 토큰의 승인을 일괄 처리

2. 캐싱 전략

- **가격 캐싱**: 1분 간격으로 가격 정보 캐시 업데이트
- **잔액 캐싱**: 유동성 확인을 위한 잔액 정보 캐싱
- **메타데이터 캐싱**: 토큰 정보 및 체인 설정 캐싱

3. 메트릭 및 모니터링

- **처리된 주문 수**: 체인별, 상태별 주문 처리 통계
- **거부된 주문 수**: 거부 사유별 통계

- **가격 모니터링**: 실시간 토큰 가격 추적
- **수익성 분석**: 주문별 수익률 및 슬리피지 추적

이 시스템은 안전하고 효율적인 크로스체인 거래 처리를 위해 다층적인 검증과 최적화 메커니즘을 제공하며, 실시간 리스크 관리와 수익성 보장을 통해 안정적인 서비스 운영을 지원합니다.